PYTHON FOR KAFKA PRODUCERS & CONSUMERS + SPARK STRUCTURED STREAMING

A practical, fully-commented interview-prep reference

Big Data Engineering | PySpark | Apache Kafka

Contents

1 Why This Python Matters for Kafka & Spark

Kafka and Spark Structured Streaming are Java/Scala systems at their core, but in real Big Data teams almost all producer/consumer glue code, ingestion scripts, and PySpark streaming jobs are written in Python. Interviewers expect you to be fluent in the specific Python patterns below — not general Python syntax.

- Writing a **producer** that serializes Python objects (dict/JSON) and pushes them onto a Kafka topic.
- Writing a **consumer** that polls a topic, deserializes messages, and processes them reliably (offsets, commits, retries).
- Using **PySpark Structured Streaming** to treat a Kafka topic as an unbounded DataFrame, transform it, and write results to a sink (console, Parquet, another Kafka topic, a database).
- Understanding how Python's role differs at each layer: raw Kafka client (confluent-kafka / kafka-python) vs. Spark's own Kafka connector (which runs on the JVM but is driven by PySpark code).

KEY DISTINCTION: Spark does NOT use the Python Kafka client under the hood. When you call `spark.readStream.format("kafka")`, Spark's JVM engine talks to Kafka directly via the spark-sql-kafka connector JAR. Python (PySpark) only defines the pipeline; it never touches raw Kafka bytes itself in Structured Streaming.

2 Python Prerequisites Checklist

Before touching Kafka or Spark streaming code, you should be comfortable with the following core Python concepts — these appear constantly in producer/consumer and streaming code:

Concept	Why it matters here
Dictionaries & `json.dumps` / `json.loads`	Kafka messages are bytes. You constantly convert Python dict <-> JSON string <-> bytes.
String encode/decode (`.encode('utf-8')`)	Kafka clients send/receive raw bytes, not str objects. Every key/value must be encoded before sending, decoded after receiving.
Functions & callbacks	Producers use a `delivery callback` function to confirm/report success or failure asynchronously.
Loops & `try/except`	Consumers run infinite `while True` polling loops; exceptions must be caught so the loop doesn't crash.
Context managers (`with`)	Used for file handles when checkpointing, and for cleanly managing Spark sessions.
Lambda functions	PySpark UDFs and simple column transformations are often written as small lambdas or one-line functions.
Classes / OOP basics	The Kafka Producer/Consumer objects are Python classes with methods like `.produce()`, `.poll()`, `.commit()`.
Environment variables / config dicts	Both Kafka clients and SparkSession are configured via Python dictionaries of key-value settings (bootstrap servers, group id, etc.).

3 Kafka Producers in Python (confluent-kafka)

The `confluent-kafka` library is the most common production-grade Python client (thin wrapper over librdkafka). Install with `pip install confluent-kafka`.

3.1 Minimal JSON Producer

```
# --- Import the Producer class from the confluent-kafka library ---
from confluent_kafka import Producer
import json

# --- Configuration dictionary: tells the producer which Kafka broker(s) to talk to ---
conf = {
"bootstrap.servers": "localhost:9092", # comma-separated list of broker host:port
"client.id": "python-producer-1" # optional, useful for logging/monitoring
}

# --- Instantiate the producer with the config above ---
producer = Producer(conf)


def delivery_report(err, msg):
"""
Callback fired by the Kafka client (asynchronously) once a message
has been delivered to the broker or has permanently failed.
'err' -> None on success, or a KafkaError object on failure
'msg' -> the original Message object, useful for logging topic/partition/offset
"""
if err is not None:
print(f"Delivery failed for record {msg.key()}: {err}")
else:
print(f"Record {msg.key()} delivered to "
f"{msg.topic()} [partition {msg.partition()}] at offset {msg.offset()}")


def send_event(topic, key, value_dict):
"""
Serializes a Python dict to JSON bytes and sends it to the given topic.
"""
# json.dumps converts the dict -> JSON string; .encode('utf-8') converts str -> bytes
payload = json.dumps(value_dict).encode("utf-8")
key_bytes = str(key).encode("utf-8")

# .produce() is asynchronous: it queues the message and returns immediately.
# The delivery_report callback runs later, once the broker acknowledges.
producer.produce(
topic=topic,
key=key_bytes,
value=payload,
callback=delivery_report
)

# .poll(0) processes any pending delivery-report callbacks without blocking.
# Must be called periodically or callbacks never fire.
producer.poll(0)


# --- Example usage: send 5 sample order events ---
for i in range(5):
order_event = {"order_id": i, "customer_id": 100 + i, "amount": 19.99 + i}
send_event("orders_topic", key=order_event["order_id"], value_dict=order_event)

# --- flush() blocks until all queued/in-flight messages are delivered or time out ---
producer.flush(timeout=10)
```

3.2 Producer Reliability Settings Worth Knowing

Config Key	Purpose
`acks`	'all' waits for all in-sync replicas to ack -> strongest durability; '1' waits only for the leader (faster, less safe).
`enable.idempotence`	'true' prevents duplicate messages on retries (exactly-once at the producer level).
`retries` / `retry.backoff.ms`	How many times / how long to wait before retrying a failed send.
`linger.ms` & `batch.size`	Trade a small delay for bigger batches -> higher throughput.
`compression.type`	'snappy' / 'gzip' / 'lz4' reduce network and disk usage.

4 Kafka Consumers in Python (confluent-kafka)

4.1 Minimal JSON Consumer with Manual Commit

```
# --- Import the Consumer class ---
from confluent_kafka import Consumer, KafkaException, KafkaError
import json

conf = {
"bootstrap.servers": "localhost:9092",
"group.id": "orders-consumer-group", # consumers sharing a group.id split partitions
"auto.offset.reset": "earliest", # where to start if no committed offset exists
"enable.auto.commit": False # we will commit offsets manually for control
}

consumer = Consumer(conf)

# --- Subscribe to one or more topics. Rebalancing across the group happens
automatically. ---
consumer.subscribe(["orders_topic"])

try:
while True: # consumers run as long-lived polling loops
msg = consumer.poll(timeout=1.0) # wait up to 1 second for a new message

if msg is None:
continue # no message arrived in this poll cycle, loop again

if msg.error():
# _PARTITION_EOF just means we've read to the end of a partition, not a real error
if msg.error().code() == KafkaError._PARTITION_EOF:
continue
else:
raise KafkaException(msg.error())

# --- Deserialize: bytes -> JSON string -> Python dict ---
value = json.loads(msg.value().decode("utf-8"))
key = msg.key().decode("utf-8") if msg.key() else None

print(f"Consumed order {key} from partition {msg.partition()} "
f"at offset {msg.offset()}: {value}")

# --- Business logic goes here (e.g. write to a database, trigger downstream job) ---
process_order(value) # placeholder for your own processing function

# --- Manually commit the offset AFTER successful processing (at-least-once semantics)
---
consumer.commit(msg)

except KeyboardInterrupt:
print("Stopping consumer...")
finally:
# Always close the consumer to leave the group cleanly and trigger a fast rebalance
consumer.close()
```

4.2 Delivery Semantics Cheat-Sheet

Semantics	How it's achieved in Python code
At-most-once	`enable.auto.commit=True` with a short auto-commit interval -> offset may commit before processing finishes.
At-least-once	Manual `consumer.commit()` called only AFTER processing succeeds (shown above). Reprocessing possible on crash/restart.
Exactly-once	Requires transactional producer (`transactional.id`) + consumer reading with `isolation.level='read_committed'`, or writing results idempotently downstream.

5 Serialization: JSON vs Avro Payloads

JSON (shown above) is simple and human-readable but has no enforced schema and is larger on the wire. In production pipelines feeding Spark, Avro + Schema Registry is common because it enforces a contract between producers and consumers and is compact.

5.1 Avro Producer Sketch (schema-registry client)

```
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
from confluent_kafka.serialization import SerializationContext, MessageField
from confluent_kafka import Producer

# --- Connect to the schema registry that stores the Avro schema definitions ---
schema_registry_conf = {"url": "http://localhost:8081"}
schema_registry_client = SchemaRegistryClient(schema_registry_conf)

order_schema_str = """
{
"type": "record",
"name": "Order",
"fields": [
{"name": "order_id", "type": "int"},
{"name": "customer_id", "type": "int"},
{"name": "amount", "type": "double"}
]
}
"""

# --- AvroSerializer converts a Python dict into Avro-encoded bytes using the schema ---
avro_serializer = AvroSerializer(schema_registry_client, order_schema_str)

producer = Producer({"bootstrap.servers": "localhost:9092"})

order = {"order_id": 1, "customer_id": 101, "amount": 42.50}

# SerializationContext tells the serializer which topic/field this is for
producer.produce(
topic="orders_avro_topic",
value=avro_serializer(order, SerializationContext("orders_avro_topic",
MessageField.VALUE))
)
producer.flush()
```

INTERVIEW TIP: In interviews: know that JSON = flexible/no schema enforcement, Avro/Protobuf = compact + schema-enforced + evolvable (backward/forward compatibility rules), which is why regulated or high-volume pipelines prefer Avro before data lands in Spark.

6 PySpark Structured Streaming: Reading from Kafka

Structured Streaming treats a Kafka topic as an unbounded, continuously-appending DataFrame. You write ordinary DataFrame code; Spark handles polling Kafka, tracking offsets, and re-running micro-batches under the hood.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import StructType, StructField, IntegerType, DoubleType

# --- Create (or get) a SparkSession; the kafka package coordinate pulls in the
connector JAR ---
spark = (
SparkSession.builder
.appName("KafkaOrdersStreamingApp")
.config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0")
.getOrCreate()
)

# --- readStream: returns a *streaming* DataFrame, not a static one ---
raw_stream_df = (
spark.readStream
.format("kafka") # use the built-in Kafka source
.option("kafka.bootstrap.servers", "localhost:9092")
.option("subscribe", "orders_topic") # topic(s) to read from, comma-separated
.option("startingOffsets", "earliest") # 'earliest' (backfill) or 'latest' (only new)
.load()
)

# --- Kafka rows arrive with fixed columns: key, value, topic, partition, offset,
timestamp ---
# 'value' is raw bytes containing our JSON payload, so we must cast + parse it.

# Define the expected schema of the JSON payload we produced earlier
order_schema = StructType([
StructField("order_id", IntegerType()),
StructField("customer_id", IntegerType()),
StructField("amount", DoubleType()),
])

parsed_df = (
raw_stream_df
.selectExpr("CAST(key AS STRING) AS order_key", "CAST(value AS STRING) AS json_value")
# from_json parses the JSON string column into structured columns per our schema
.withColumn("data", from_json(col("json_value"), order_schema))
.select("order_key", "data.*") # flatten the nested struct into top-level columns
)

# --- Write the parsed stream to the console for debugging (a "sink") ---
query = (
parsed_df.writeStream
.format("console")
.outputMode("append") # 'append' = only show new rows since last trigger
.option("truncate", False)
.start()
)

query.awaitTermination() # keeps the streaming job alive until manually stopped
```

7 PySpark Structured Streaming: Writing to Sinks / Kafka

The same streaming DataFrame can be written to many sink types: console (debug), Parquet files (bronze/silver/gold layers), a database via foreachBatch, or back into another Kafka topic for downstream consumers.

7.1 Writing Back to a Kafka Topic

```
from pyspark.sql.functions import to_json, struct, col

# --- Kafka sink requires a 'value' column (and optionally 'key'), both as strings/bytes
---
output_df = parsed_df.select(
col("order_key").alias("key"),
to_json(struct("order_id", "customer_id", "amount")).alias("value")
)

kafka_sink_query = (
output_df.writeStream
.format("kafka")
.option("kafka.bootstrap.servers", "localhost:9092")
.option("topic", "orders_enriched_topic")
# checkpointLocation stores offset + state so the job can resume exactly where it left
off
.option("checkpointLocation", "/tmp/checkpoints/orders_enriched")
.outputMode("append")
.start()
)

kafka_sink_query.awaitTermination()
```

7.2 Writing to Parquet (Bronze/Silver Medallion Layer)

```
parquet_query = (
parsed_df.writeStream
.format("parquet")
.option("path", "/data/bronze/orders") # where the Parquet files land
.option("checkpointLocation", "/tmp/checkpoints/orders_bronze")
.outputMode("append") # Parquet sink only supports append mode
.trigger(processingTime="30 seconds") # micro-batch every 30 seconds
.start()
)
parquet_query.awaitTermination()
```

8 Windowing, Watermarking & Aggregations

Real interview questions almost always probe windowed aggregations and late-data handling. Watermarking tells Spark how long to wait for late-arriving events before finalizing (and dropping state for) a window.

```
from pyspark.sql.functions import window, sum as spark_sum, col

# Assume 'parsed_df' also has an 'event_time' timestamp column from the payload
windowed_revenue = (
parsed_df
# withWatermark: tolerate up to 10 minutes of late data based on 'event_time'
.withWatermark("event_time", "10 minutes")
.groupBy(
window(col("event_time"), "5 minutes"), # tumbling 5-minute windows
col("customer_id")
)
.agg(spark_sum("amount").alias("total_amount"))
)

query = (
windowed_revenue.writeStream
.format("console")
.outputMode("update") # 'update' -> only rows that changed since last trigger are shown
.option("truncate", False)
.start()
)
query.awaitTermination()
```

Output Mode	Meaning
append	Only new finalized rows are emitted; cannot be used with aggregations unless watermarked.
update	Only rows whose aggregate value changed since the previous trigger are emitted.
complete	The entire updated result table is re-emitted every trigger (small aggregations only).

9 foreachBatch: Custom Sink Logic (Medallion Pattern)

`foreachBatch` gives you the underlying micro-batch as a regular (static) DataFrame, so you can reuse ordinary batch PySpark code — write to multiple sinks, upsert into a Delta/SQL table, or apply the medallion bronze -> silver -> gold pattern inside a streaming job.

```
def process_batch(batch_df, batch_id):
"""
Called once per micro-batch. 'batch_df' behaves like a normal (non-streaming)
DataFrame here, so all standard batch transformations/writes are allowed.
"""
print(f"Processing batch #{batch_id} with {batch_df.count()} rows")

# --- Example: silver layer -> clean + deduplicate ---
silver_df = batch_df.dropDuplicates(["order_id"]).filter(batch_df.amount > 0)

# --- Write silver layer to Parquet ---
silver_df.write.mode("append").parquet("/data/silver/orders")

# --- Example: also write a gold aggregate straight to a database (JDBC) ---
gold_df = silver_df.groupBy("customer_id").sum("amount")
(
gold_df.write
.format("jdbc")
.option("url", "jdbc:postgresql://localhost:5432/warehouse")
.option("dbtable", "gold_customer_totals")
.option("user", "etl_user")
.option("password", "etl_password")
.mode("overwrite")
.save()
)


batch_query = (
parsed_df.writeStream
.foreachBatch(process_batch) # our custom function runs on every micro-batch
.option("checkpointLocation", "/tmp/checkpoints/orders_foreachbatch")
.start()
)
batch_query.awaitTermination()
```

10 End-to-End Mini Pipeline

This section shows how sections 3-9 fit together into a single flow you can describe clearly in an interview:

- **producer.py** — a Python script using confluent-kafka to publish JSON order events onto `orders_topic`.
- **Kafka broker** — durably stores the events across partitions, replicated for fault tolerance.
- **spark_streaming_job.py** — a PySpark Structured Streaming application that reads from `orders_topic`, parses JSON, applies windowed aggregation, and uses `foreachBatch` to write bronze/silver/gold outputs.
- **Sinks** — Parquet data lake (bronze/silver), a data warehouse table (gold), and optionally a re-published Kafka topic for other downstream consumers.

```
# run_pipeline.sh (conceptual orchestration - for interview explanation only)

# 1. Start the Python producer (keeps running / can be scheduled)
python3 producer.py

# 2. Submit the PySpark Structured Streaming job to a Spark cluster
spark-submit \
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0 \
spark_streaming_job.py

# 3. Both processes run continuously:
# producer.py keeps publishing events -> Kafka retains them -> the Spark job
# keeps consuming, transforming, and writing to bronze/silver/gold + optional Kafka
sink.
```

11 Quick Reference Tables

11.1 confluent-kafka Producer API

Method	Purpose
`Producer(conf)`	Create a producer instance from a config dict.
`.produce(topic, key, value, callback)`	Queue a message for async delivery.
`.poll(timeout)`	Serve delivery-report callbacks; must be called periodically.
`.flush(timeout)`	Block until all queued messages are sent or timeout elapses.

11.2 confluent-kafka Consumer API

Method	Purpose
`Consumer(conf)`	Create a consumer instance from a config dict.
`.subscribe([topics])`	Join a consumer group and subscribe to topic(s).
`.poll(timeout)`	Fetch the next available message, or None.
`.commit(msg)`	Manually commit the offset for a processed message.
`.close()`	Cleanly leave the consumer group.

11.3 PySpark Structured Streaming Essentials

API	Purpose
`spark.readStream.format('kafka')`	Create a streaming source DataFrame from a Kafka topic.
`.writeStream.format(...)`	Define a sink: console, kafka, parquet, foreachBatch, etc.
`.outputMode(...)`	append / update / complete — controls what gets emitted each trigger.
`.trigger(processingTime=...)`	Set the micro-batch interval.
`.option('checkpointLocation', path)`	Persist offsets/state so the job can resume after a failure.
`withWatermark(col, delay)`	Bound how long Spark waits for late data before finalizing a window.
`.foreachBatch(func)`	Run ordinary batch DataFrame code against each micro-batch.
`query.awaitTermination()`	Blocks the driver, keeping the streaming query alive.

12 Common Interview Questions & Answers

Q: Why must Kafka message keys/values be encoded to bytes in Python?

A: Kafka's wire protocol only understands raw bytes; the client library does not assume a text encoding for you, so Python str objects must be explicitly '.encode("utf-8")'d before sending and '.decode("utf-8")'d after receiving.

Q: What's the difference between 'poll()' in the Kafka Python consumer and 'poll()' in the producer?

A: Consumer.poll() fetches the next available message from the subscribed topic(s). Producer.poll() does not fetch data; it services the producer's internal event queue so that delivery-report callbacks for already-sent messages get invoked.

Q: Why does Structured Streaming need a checkpoint location?

A: The checkpoint directory stores processed Kafka offsets and any stateful aggregation data. If the job crashes and restarts, Spark reads the checkpoint to resume exactly where it left off instead of reprocessing or skipping data.

Q: When would you choose 'foreachBatch' over a native sink like 'format("parquet")'?

A: When you need logic a native sink can't express: writing to multiple destinations, upserts/merges (e.g. Delta MERGE), deduplication, or calling non-Spark-native APIs (JDBC writes, REST calls) per micro-batch.

Q: How do you avoid duplicate processing in a Kafka consumer if it crashes mid-batch?

A: Commit offsets manually only after the downstream write succeeds (at-least-once), and make the downstream write idempotent (e.g. upsert on a unique key) so re-processing the same message twice doesn't create duplicate records.

End of guide — pair this with your PySpark transformation reference and Spark Structured Streaming slide deck for full interview coverage.